PTFS-CAM PROJECT REPORT SS - 2022

2D Modeling of the steady-state heat equation

Spondon Sarker

(Matriculation No –**22733108[ptfs166h]**)

September 02, 2022

Table of Contents:

Part 1: Roofline calculation for CG, PCG

Expected performance conclusions: applyStencil, axpby, dot, GSPreCon

Expected performance conclusions: CG, PCG

Part 2: Comparison of real results vs expected (1 socket = 18 cores).

Part 3: Code balance measurement: **applyStencil, axpby, CGPreCon**.

Part 4: Comparison of scaling the code from 18 cores to 72 cores.

Part 1: Calculate the roofline predictions for CG and PCG solvers on one socket (18 cores) of Fritz. Calculate for two grid sizes: 1000 x 20 000 and 1000 x 100000. Does the performance change? Why?

I parallelized the functions and subroutines involved in the CG and PCG solvers (from **Solver.cpp**). This includes **applyStencil**, **dotProduct**, **axpby**, **GSPreCon** and several subroutines from **Grid.cpp**. Apart from that, the corresponding directives **(#pragma omp parallel** and **#pragma omp parallel for**), I used **schedule(static)** and **reduction** (when needed).

The roofline predictions for the subroutines of the CG and PCG solvers are given in [LUP/s] and I consider the execution in single socket of Fritz (i.e. 18 cores). It was done for two grid sizes: 2000×20000 and 1000×400000. The number of cycles per instruction was based on **Intel ICE LAKE** processor. Also, I took the memory bandwidth of 1 ccNUMA domain of Fritz to be 76 GB/s according to the instruction.

As we are working with stencils that is why first I compared the given sizes with the L3 cache size in order to determine whether the **Layer Condition** was fulfilled. For this, I used the formula:

$$\textbf{Layer Condition} = \boldsymbol{xSize} \times \mathbf{8}\ \boldsymbol{Bytes}\ \times \boldsymbol{nthreads}\ \times \mathbf{3}(\#\boldsymbol{rows})\ \leq \frac{\boldsymbol{L3\ cache\ size}}{\mathbf{2}}$$

The **L3 cache size = 54 Mbytes** and **nthreads = 18**(1 socket = 18 cores).

<u>For (xSize = 20000):</u>

$$\textbf{Layer Condition} = 2 \times 10^4\ \times 8\ \times 18\ \times 3\ \leq \frac{54 \times 10^6}{2}$$

$$\Rightarrow 1728\ \leq 5400$$

So the **Layer condition holds**. Data fits into L3 Cache.

<u>For (xSize = 400000):</u>

$$\textbf{Layer Condition} = 4 \times 10^5\ \times 8\ \times 18\ \times 3\ \leq \frac{54 \times 10^6}{2}$$

$$\Rightarrow 3456 \leq 540$$

Hence, this violates the **Layer Condition**. Data must be loaded from Main Memory.

The next roofline predictions are of the functions within the 'while' loop of CG and PCG routines.

I. **applyStencil (lhs, x):** From the kernel

```
for ( int j=1; j<ySize-1; ++j)
    {
        for (int i=1; i<xSize-1; ++i)
        {
            (*lhs)(j,i) = w_c*(*x)(j,i) - w_y*((*x)(j+1,i) + (*x)(j-1,i))
- w_x*((*x)(j,i+1) + (*x)(j,i-1));

        }
    }
```

For (xSize = 20000)

It is a 7 flop operations (4 ADD, 3 MULT), 2 LOAD, 1 STORE. **Intel Ice Lake – AVX512** core has maximum of 5 instructions per cycle. So, here we have the bottleneck of ADD. Which corresponds to a lattice site update (LUP). With the formula:

$$\boldsymbol{P_{max} = C\,(\frac{Flops\ per\ AVX_iteration}{cycle}) \times n_{SIMD} \times f \times nthreads}$$

$$(C = \frac{LUP}{cy}, f = \boldsymbol{frequency\ of\ the\ processor}, n_{SIMD} = \boldsymbol{vectorization\ factor})$$

So, the maximum attainable performance is:

$$\boldsymbol{P_{max}} = \frac{1}{2} \times 8 \times 2 \times 10^9 \times 18 = 144\ GLUP/s$$

Here, $\boldsymbol{n_{SIMD} = 8}$.

The data traffic for one update consist of 2 LOAD and 1 STORE. All elements are double floating-point numbers, hence

$$I = \frac{1}{B_C}, B_C = 3 \times \frac{8B}{1\ LUP}$$

($I = \boldsymbol{intensity}$ *and* $B_C = \boldsymbol{Memory\ code\ balance}$). As in the CAM tutorials the instruction was to take the memory bandwidth of Fritz to be $\boldsymbol{b_s}$ **=76 GB/s** and then

$$I \times b_s = \frac{76}{24}\frac{GLUP}{s}.$$

From the roofline model it follows:

$$\boldsymbol{P_{Stencil}} = \min(P_{max}, I \times b_s) = \min(144, 3.16)\frac{GLUP}{s} = 3.16\frac{GLUP}{s}$$

For (xSize = 400000)

Here, the layer condition does not hold anymore, data must be loaded from main memory. So, the maximum attainable performance changes into:

$$\boldsymbol{P_{max}} = \frac{1}{2} \times 8 \times 2 \times 10^9 \times 18 = 144\ GLUP/s$$

And the **Computational Intensity** changes into:

$$I = \frac{1}{B_C}, B_C = 5 \times \frac{8B}{1\ LUP}$$

$$I \times b_s = \frac{76}{40}\frac{GLUP}{s}.$$

From the roofline model it follows:

$$\boldsymbol{P_{Stencil}} = \min(P_{max}, I \times b_s) = \min(144, 1.9)\frac{GLUP}{s} = 1.9\frac{GLUP}{s}$$

II. axpby(lhs[:] = a*x[:] + b*y[:]): from the kernel

```
 for(int yIndex=shift; yIndex<lhs->numGrids_y(true)-shift; ++yIndex)
    {
        for(int xIndex=shift; xIndex<lhs->numGrids_x(true)-shift;
++xIndex)
        {
            (*lhs)(yIndex,xIndex) = (a*(*x)(yIndex,xIndex)) +
(b*(*y)(yIndex,xIndex));
        }
    }
```

For the both cases **(xSize = 20000 and xSize = 400000):**

Here, LOAD: x[:], y[:]

STORE: lhs[:] (either x[:] or y[:] – already loaded – no need to LOAD)

No clear bottlenecks here and it takes only one cycle to execute the kernel once.

Now the **Maximum Attainable Performance** is:

$$\boldsymbol{P_{max}} = 1 \times 8 \times 2 \times 10^9 \times 18 = 288\ GLUP/s$$

And the **Computational Intensity** changes into: 2 LOAD and 1 STORE

$$I = \frac{1}{B_C}, B_C = 3 \times \frac{8B}{1\ LUP}$$

$$I \times b_s = \frac{76}{24}\frac{GLUP}{s}.$$

From the roofline model it follows:

$$\boldsymbol{P_{axpby}} = \min(P_{max}, I \times b_s) = \min(288, 3.16)\frac{GLUP}{s} = 3.16\frac{GLUP}{s}$$

III. **dotProduct:** from the kernel

```
for(int yIndex=shift; yIndex<x->numGrids_y(true)-shift; ++yIndex)
   {
       for(int xIndex=shift; xIndex<x->numGrids_x(true)-shift; ++xIndex)
       {
           dot_res += (*x)(yIndex,xIndex)*(*y)(yIndex,xIndex);
       }
   }
```

For ($\boldsymbol{x \neq y\ with\ xSize = 20000\ or\ xSize = 400000}$):

There is no clear bottleneck here and it takes only 1 cycle to execute the kernel once.

So the **Maximum Attainable** performance is:

$$\boldsymbol{P_{max}} = 1 \times 8 \times 2 \times 10^9 \times 18 = 288\ GLUP/s$$

And the **Computational Intensity** changes into: 2 Flops (1 MULT, 1 ADD) and 2 LOAD

$$I = \frac{1}{B_C}, B_C = 2 \times \frac{8B}{1\ LUP}$$

$$I \times b_s = \frac{76}{16}\frac{GLUP}{s}.$$

From the roofline model it follows:

$$\boldsymbol{P_{dotproduct(diff)}} = \min(P_{max}, I \times b_s) = \min(288, 4.75)\frac{GLUP}{s} = 4.75\frac{GLUP}{s}$$

For ($x = y$ *with* $xSize = 20000$ *or* $xSize = 400000$):

Like before there is no clear bottleneck here and it takes only one cycle to execute the kernel once.

So the **Maximum Attainable** performance is:

$$\boldsymbol{P_{max}} = 1 \times 8 \times 2 \times 10^9 \times 18 = 288\ GLUP/s$$

And the **Computational Intensity** changes into: 1 LOAD

$$I = \frac{1}{B_C}, B_C = 1 \times \frac{8B}{1\ LUP}$$

$$I \times b_s = \frac{76}{8}\frac{GLUP}{s}.$$

From the roofline model it follows:

$$\boldsymbol{P_{dotproduct(same)}} = \min(P_{max}, I \times b_s) = \min(288, 9.5)\frac{GLUP}{s} = 9.5\frac{GLUP}{s}$$

IV. GSPreCon:

```
 //FORWARD SUBSTITUTION
for ( int j=1; j<ySize-1+nthreads-1; ++j)
    {
       for ( int i=1; i<xSize-1; ++i)
          {
              (*x)(j,i) = w_c*((*rhs)(j,i) + (w_y*(*x)(j-1,i) +
w_x*(*x)(j,i-1)));
          }
     }

 //BACKWARD SUBSTITUTION
 for ( int j=ySize-2; j>0; --j)

    {
       for ( int i=xSize-2; i>0; --i)
         {
              (*x)(jb,i) = (*x)(j,i) + w_c*(w_y*(*x)(j+1,i)
+w_x*(*x)(j,i+1));
          }
    }
```

For (xSize = 20000):

As I have to work with GS that is why first I compared the given sizes with the L3 cache size in order to determine whether the layer condition was fulfilled.

$$\boldsymbol{Layer\ Condition = xSize \times 8\ Bytes\ \times nthreads\ \times 2(\#rows)\ \leq \frac{L3\ cache\ size}{2}}$$

$$2 \times 10^4\ \times 8\ \times 18\ \times 2\ \leq \frac{54 \times 10^6}{2}$$

$$\Rightarrow 1152\ \leq 5400$$

So the **Layer condition holds**. Data fits into L3 cache.

<u>Forward Substitution:</u>

```
for ( int j=1; j<ySize-1+nthreads-1; ++j)
    {
       for ( int i=1; i<xSize-1; ++i)
          {
              (*x)(j,i) = w_c*((*rhs)(j,i) + (w_y*(*x)(j-1,i) +
w_x*(*x)(j,i-1)));
          }
     }
```

Here, bottleneck is MULT (2 cycles).

So the **Maximum Attainable Performance** is:

$$\boldsymbol{P_{max}} = \frac{1}{2} \times 8 \times 2 \times 10^9 \times 18 = 144\ GLUP/s$$

And the **Computational Intensity** changes into: 2 LOAD, 1 STORE

$$I = \frac{1}{B_C}\ , B_C = 3\ \times \frac{8B}{1\ LUP}$$

$$I\ \times b_s = \frac{76}{24}\frac{GLUP}{s}.$$

From the roofline model it follows:

$$\boldsymbol{P_{PreConFw}} = \min(P_{max}, I \times b_s) = \min(144, 3.16)\frac{GLUP}{s} = 3.16\frac{GLUP}{s}$$

<u>Backward Substitution:</u>

```
for ( int j=ySize-2; j>0; --j)

    {
       for ( int i=xSize-2; i>0; --i)
         {
              (*x)(jb,i) = (*x)(j,i) + w_c*(w_y*(*x)(j+1,i)
+w_x*(*x)(j,i+1));
          }
    }
```

Here, The bottleneck is MULT (2 cycles).

So the **Maximum Attainable** performance is:

$$P_{max} = \frac{1}{2} \times 8 \times 2 \times 10^9 \times 18 = 144\ GLUP/s$$

And the **Computational Intensity** changes into: 1 LOAD, 1 STORE

$$I = \frac{1}{B_C}, B_C = 2 \times \frac{8B}{1\ LUP}$$

$$I \times b_s = \frac{76}{16}\frac{GLUP}{s}.$$

From the roofline model it follows:

$$\boldsymbol{P_{PreConBw}} = \min(P_{max}, I \times b_s) = \min(144, 4.75)\frac{GLUP}{s} = 4.75\frac{GLUP}{s}$$

The total performance is computed using a time based model. Let, $\mathbf{T_{PreConFW}}$ and $\mathbf{T_{PreConBW}}$ be the time required for the forward and backward substitution, respectively. I used the general formula $= \frac{W}{P}$, where W is the amount of workload done, T the total time of execution and P is the performance of the routine. Also set W = 1, considering one LUP.

Then,

$$T_{PreCon} = T_{PreConFW} + T_{PreConBW}$$

$$\frac{1}{P_{PreCon}} = \frac{1}{P_{PreConFW}} + \frac{1}{P_{PreConBW}}$$

$$\frac{1}{P_{PreCon}} = \frac{1}{3.16} + \frac{1}{4.75} = 0.5269 \text{ s/GLUP}$$

$$\mathbf{P_{PreCon}} = \frac{1}{0.5269} \text{ s/GLUP} = 1.8978 \text{ GLUP/s}$$

<u>For (xSize = 400000):</u>

$$\boldsymbol{Layer\ Condition} = 4 \times 10^5 \times 8 \times 18 \times 2 \leq \frac{54 \times 10^6}{2}$$

$$\Rightarrow 2304 \leq 540$$

Hence, **this violates the layer condition**. Data must be loaded from main memory.

<u>Forward Substitution:</u>

```
for ( int j=1; j<ySize-1+nthreads-1; ++j)
    {
       for ( int i=1; i<xSize-1; ++i)
          {
              (*x)(j,i) = w_c*((*rhs)(j,i) + (w_y*(*x)(j-1,i) +
w_x*(*x)(j,i-1)));
          }
     }
```

Here, The bottleneck is MULT (2 cycles).

So the **Maximum Attainable performance** is:

$$\boldsymbol{P_{max}} = \frac{1}{2} \times 8 \times 2 \times 10^9 \times 18 = 144\ GLUP/s$$

And the **Computational Intensity** changes into: 3 LOAD, 1 STORE

$$I = \frac{1}{B_C}, B_C = 4 \times \frac{8B}{1\ LUP}$$

$$I \times b_s = \frac{76}{32} \frac{GLUP}{s}.$$

From the roofline model it follows:

$$\boldsymbol{P_{PreConFw}} = \min(P_{max}, I \times b_s) = \min(144, 2.37)\frac{GLUP}{s} = 2.37\frac{GLUP}{s}$$

<u>Backward Substitution:</u>

```
for ( int j=ySize-2; j>0; --j)

    {
       for ( int i=xSize-2; i>0; --i)
         {
              (*x)(jb,i) = (*x)(j,i) + w_c*(w_y*(*x)(j+1,i)
+w_x*(*x)(j,i+1));
          }
    }
```

Here, The bottleneck is MULT (2 cycles).

So the **Maximum Attainable Performance** is:

$$\boldsymbol{P_{max}} = \frac{1}{2} \times 8 \times 2 \times 10^9 \times 18 = 144\ GLUP/s$$

And the **Computational Intensity** changes into: 2 LOAD, 1 STORE

$$I = \frac{1}{B_C}, B_C = 3 \times \frac{8B}{1\ LUP}$$

$$I \times b_s = \frac{76}{24}\frac{GLUP}{s}.$$

From the roofline model it follows:

$$\boldsymbol{P_{PreConBw}} = \min(P_{max}, I \times b_s) = \min(144, 3.16)\frac{GLUP}{s} = 3.16\frac{GLUP}{s}$$

Now the **Total Performance:**

$$T_{PreCon} = T_{PreConFW} + T_{PreConBW}$$

$$\frac{1}{P_{PreCon}} = \frac{1}{P_{PreConFW}} + \frac{1}{P_{PreConBW}}$$

$$\frac{1}{P_{PreCon}} = \frac{1}{2.37} + \frac{1}{3.16} = 0.7383 \text{ s/GLUP}$$

$$\mathbf{P_{PreCon}} = \frac{1}{0.7383} \text{ s/GLUP} = 1.35 \text{ GLUP/s}$$

<u>Conclusion on each function's performance:</u>

Grid size	$P_{Stencil}$	$P_{dotproduct(diff)}$	$P_{dotproduct(same)}$	P_{axpby}	P_{PreCon}
2000x20000	3.16 GLUP/s	4.75 GLUP/s	9.5 GLUP/s	3.16 GLUP/s	1.89 GLUP/s
1000x400000	1.9 GLUP/s				1.35 GLUP/s

<u>Total Performance of CG:</u>

To compute the performance of CG and PCG routines, we use again the time based model:

- **xSize = 20000:**

It holds

$$T_{CG} = 1 \times T_{Stencil} + 1 \times T_{dotproduct(diff)} + 1 \times T_{dotproduct(same)} + 3 \times T_{axpby}$$

$$\frac{1}{P_{CG}} = \frac{1}{P_{Stencil}} + \frac{1}{P_{dotproduct(diff)}} + \frac{1}{P_{dotproduct(same)}} + 3 \times \frac{1}{P_{axpby}}$$

$$\frac{1}{P_{CG}} = \frac{1}{3.16} + \frac{1}{4.75} + \frac{1}{9.5} + 3 \times \frac{1}{3.16}$$

$$\frac{1}{P_{CG}} = 1.58 \frac{s}{GLUP}$$

$$\boldsymbol{P_{CG}} = 0.63291 \frac{GLUP}{s} = 632.91 \frac{MLUP}{s}$$

- **xSize = 400000:**

It holds

$$\boldsymbol{T_{CG} = 1 \times T_{Stencil} + 1 \times T_{dotproduct(diff)} + 1 \times T_{dotproduct(same)} + 3 \times T_{axpby}}$$

$$\frac{1}{P_{CG}} = \frac{1}{P_{Stencil}} + \frac{1}{P_{dotproduct(diff)}} + \frac{1}{P_{dotproduct(same)}} + 3 \times \frac{1}{P_{axpby}}$$

$$\frac{1}{P_{CG}} = \frac{1}{1.9} + \frac{1}{4.75} + \frac{1}{9.5} + 3 \times \frac{1}{3.16}$$

$$\frac{1}{P_{CG}} = 1.79 \frac{s}{GLUP}$$

$$\boldsymbol{P_{CG}} = 0.55865 \frac{GLUP}{s} = 558.65 \frac{MLUP}{s}$$

<u>Total Performance of PCG:</u>

- **xSize = 20000:**

It holds

$$\boldsymbol{T_{PCG} = 1 \times T_{Stencil} + 2 \times T_{dotproduct(diff)} + 1 \times T_{dotproduct(same)} + 3 \times T_{axpby} + T_{PreCon}}$$

$$\frac{1}{P_{PCG}} = \frac{1}{P_{Stencil}} + \frac{2}{P_{dotproduct(diff)}} + \frac{1}{P_{dotproduct(same)}} + 3 \times \frac{1}{P_{axpby}} + \frac{1}{P_{PreCon}}$$

$$\frac{1}{P_{PCG}} = \frac{1}{3.16} + \frac{2}{4.75} + \frac{1}{9.5} + 3 \times \frac{1}{3.16} + \frac{1}{1.89}$$

$$\frac{1}{P_{PCG}} = 2.32 \frac{s}{GLUP}$$

$$\boldsymbol{P_{PCG}} = 0.43103 \frac{GLUP}{s} = 431.03 \frac{MLUP}{s}$$

- **xSize = 400000:**

It holds

$$\boldsymbol{T_{PCG} = 1 \times T_{Stencil} + 2 \times T_{dotproduct(diff)} + 1 \times T_{dotproduct(same)} + 3 \times T_{axpby} + T_{PreCon}}$$

$$\frac{1}{P_{PCG}} = \frac{1}{P_{Stencil}} + \frac{2}{P_{dotproduct(diff)}} + \frac{1}{P_{dotproduct(same)}} + 3 \times \frac{1}{P_{axpby}} + \frac{1}{P_{PreCon}}$$

$$\frac{1}{P_{PCG}} = \frac{1}{1.9} + \frac{2}{4.75} + \frac{1}{9.5} + 3 \times \frac{1}{3.16} + \frac{1}{1.35}$$

$$\frac{1}{P_{PCG}} = 2.74 \frac{s}{GLUP}$$

$$\boldsymbol{P_{PCG}} = 0.36496 \frac{GLUP}{s} = 364.96 \frac{MLUP}{s}$$

Expected Performance of Solvers:

Grid Size	2000 x 20000	1000 x 400000
CG (MLUP/s)	632.91	558.65
PCG (MLUP/s)	431.03	364.96

Performance is expected to drop, because **xSize = 400000** violates the layer condition (does not fit into L3 cache) and therefore, more data needs to be loaded from the main memory (which is slower, hence, less work may be performed at the same time).

Part 2: Check whether you attain the roofline performance by running the code on 1 socket of Fritz. Timings are already included. Run the code using following command: ./perf num_grids_y num_grids_x.

To validate the predictions, I measured the performance in Fritz. I worked with my username **ptfs166h** and requested a single interactive node:

ptfs166h@fritz3:~/ptfsProject$ salloc --nodes=1 --time=1:00:00 -p singlenode -C hwperf

Then, I compiled the code:

module load intel
module load likwid
LIKWID =on CXX = icpc make

To measure performance on 1 socket with setting frequency **2GHz**, I run the binary perf using LIKWID:

ptfs166h@fritz2:~/ptfsProject$ srun --cpu-freq=2000000-2000000 likwid-pin -C S0:0-17 ./perf 2000 20000

ptfs166h@fritz2:~/ptfsProject$ srun --cpu-freq=2000000-2000000 likwid-pin -C S0:0-17 ./perf 1000 400000

The results are shown in the pictures. It is relevant to remark that the threads were pinned to the cores using **likwid-pin** (this corresponds to compact pinning) and that I used only one socket of Fritz. The frequency of the corresponding node was fixed at 2 GHz.

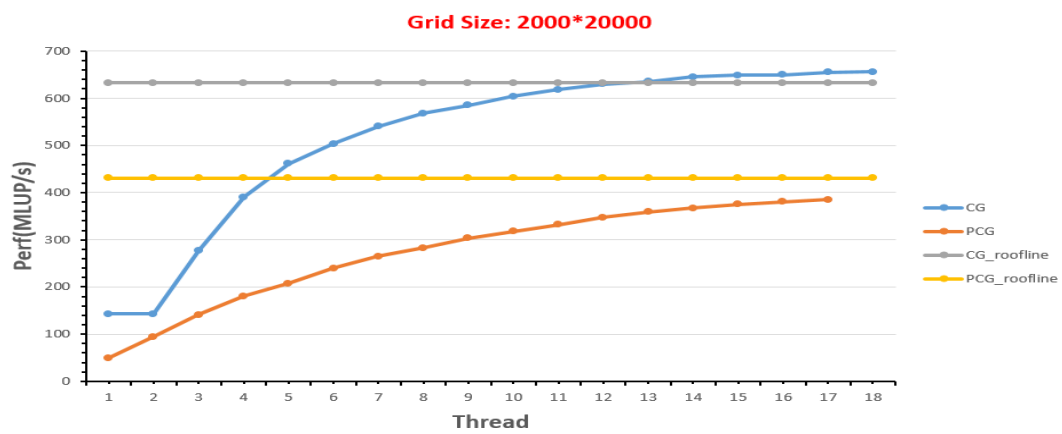
Real Performance results

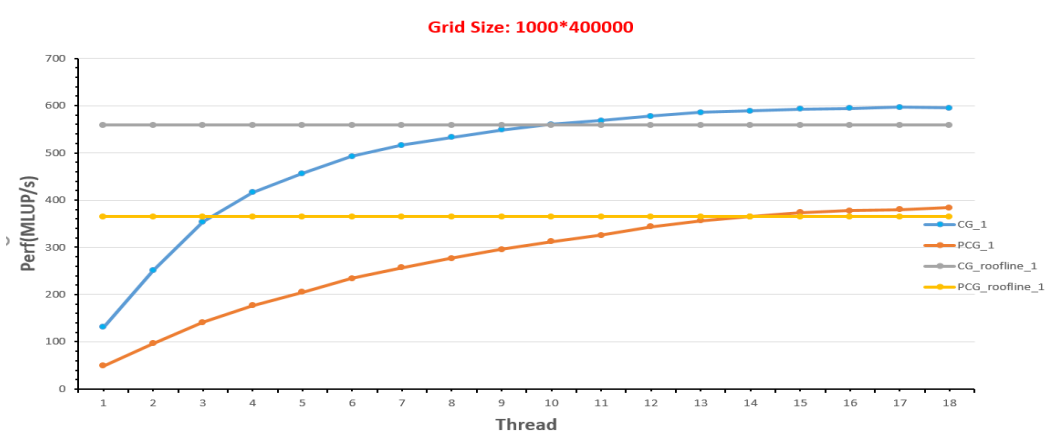
Grid Size	2000 x 20000	1000 x 400000
CG (MLUP/s)	664.437	594.782
PCG (MLUP/s)	389.113	384.114

Let us assemble the **expected/real results** into one table:

	Real	Expected	Real	Expected
Grid Size	2000 x 20000		1000 x 400000	
CG (MLUP/s)	665	633	595	559
PCG (MLUP/s)	389	431	385	365

As we can see from the measurements, my assumptions on the roofline reflect the real performance pretty well. Only difference in CG(Real) vs. CG(expected), where real performance exceeds expected (which might be due to small amount of tests and/or relatively small grid sizes).





Part 3: Measure the code balance in (B/LUP) of applyStencil kernel on one socket (18 threads) for both grid dimensions and comment whether it agrees to you model.

With the help of LIKWID, I can have a report of the amount of calls and the data traffic regarding **applyStencil** and **GSPreCon**. Markers are inserted in the code and for the routines of interest they have to be included in the corresponding parallel sections. Then, the binary is executed with the additional command **likwid-perfctr** (with the necessary flags):

ptfs166h@f0971:~/ptfsProject$ srun --cpu-freq=2000000-2000000 likwid-perfctr -C S0:0-17 -g MEM -m ./perf 2000 20000

ptfs166h@f0971:~/ptfsProject$ srun --cpu-freq=2000000-2000000 likwid-perfctr -C S0:0-17 -g MEM -m ./perf 1000 400000

The output is a list with several data and measurements, but I only need the call count and the volume of data traffic, so I compute the **code balance** as:

$$B_C = \frac{\boldsymbol{data\ volume}}{\boldsymbol{call\ count}\ \times (\boldsymbol{xSize} - 2) \times (\boldsymbol{ySize} - 2)}$$

(case xSize = 20000)

For **applyStencil,** I obtain a total amount of data volume **38.01 GB** for **45 calls**. Hence, the code balance is:

$$\frac{30.01 \times 10^9\ bytes}{45 \times 19998 \times 1998} \approx 21\ \frac{B}{LUP}$$

Comparing this number with our computation of **24 B/LUP**, is nearly accurate.

```
Region APPLY_STENCIL, Group 1: MEM
+-------------------+------------+-
--+-------------+-------------+----
|    Region Info    | HWThread 0 |
2 | HWThread 13 | HWThread 14 | HWT
+-------------------+------------+-
--+-------------+-------------+----
| RDTSC Runtime [s] |   0.666137 |
2 |    0.475944 |    0.470850 |
|     call count    |         45 |
5 |          45 |          45 |
+-------------------+------------+-
```

```
+-----------------------------------+------------+
----+-------------+-------------+-------------+---
|               Metric              | HWThread 0 |
 11 | HWThread 12 | HWThread 13 | HWThread 14 | HWT
+-----------------------------------+------------+
----+-------------+-------------+-------------+---
|        Runtime (RDTSC) [s]        |     0.6661 |
866 |      0.4757 |      0.4759 |      0.4708 |
|        Runtime unhalted [s]       |     0.5275 |
023 |      0.3934 |      0.3933 |      0.3894 |
|           Clock [MHz]             |  1995.2674 |
612 |   1995.2634 |   1995.2607 |   1995.2594 |
|               CPI                 |     2.2542 |
118 |      3.5750 |      3.3994 |      3.6895 |
|  Memory read bandwidth [MBytes/s] | 33083.9529 |
  0 |           0 |           0 |           0 |
|  Memory read data volume [GBytes] |    22.0384 |
  0 |           0 |           0 |           0 |
| Memory write bandwidth [MBytes/s] | 23984.2234 |
  0 |           0 |           0 |           0 |
| Memory write data volume [GBytes] |    15.9768 |
  0 |           0 |           0 |           0 |
|    Memory bandwidth [MBytes/s]    | 57068.1763 |
  0 |           0 |           0 |           0 |
|    Memory data volume [GBytes]    |    38.0152 |
  0 |           0 |           0 |           0 |
+-----------------------------------+------------+
----+-------------+-------------+-------------+---
```

Call count and data volume of applyStencil for xSize = 20000

For **GSPreCon** I obtain a total amount of data volume of **32.21 GB** for **21 calls**. Hence, the code balance is:

$$\frac{32.21 \times 10^9\ bytes}{21 \times 19998 \times 1998} \approx 38\ \frac{B}{LUP}$$

Comparing this number with our computation of **40 B/LUP**, is nearly accurate.

```
Region GS_PRE_CON, Group 1: MEM
+-------------------+------------+
--+-------------+-------------+---
|    Region Info    | HWThread 0 |
2 | HWThread 13 | HWThread 14 | HW
+-------------------+------------+
--+-------------+-------------+---
| RDTSC Runtime [s] |   0.719957 |
4 |    0.725259 |    0.725242 |
|     call count    |         21 |
1 |          21 |          21 |
+-------------------+------------+
--+-------------+-------------+---
```

```
+-----------------------------------+------------+
----+-------------+-------------+-------------+---
|                Metric             | HWThread 0 |
 11 | HWThread 12 | HWThread 13 | HWThread 14 | HW
+-----------------------------------+------------+
----+-------------+-------------+-------------+---
|        Runtime (RDTSC) [s]        |     0.7200 |
253 |      0.7253 |      0.7253 |      0.7252 |
|        Runtime unhalted [s]       |     0.5957 |
017 |      0.6016 |      0.6016 |      0.6015 |
|            Clock [MHz]            |  1995.2645 |
610 |   1995.2622 |   1995.2640 |   1995.2640 |
|                  CPI              |     1.6440 |
869 |      1.5993 |      1.6388 |      1.5980 |
|  Memory read bandwidth [MBytes/s] | 27666.0394 |
  0 |           0 |           0 |           0 |
|  Memory read data volume [GBytes] |    19.9184 |
  0 |           0 |           0 |           0 |
| Memory write bandwidth [MBytes/s] | 17085.5179 |
  0 |           0 |           0 |           0 |
| Memory write data volume [GBytes] |    12.3008 |
  0 |           0 |           0 |           0 |
|    Memory bandwidth [MBytes/s]    | 44751.5574 |
  0 |           0 |           0 |           0 |
|    Memory data volume [GBytes]    |    32.2192 |
  0 |           0 |           0 |           0 |
+-----------------------------------+------------+
----+-------------+-------------+-------------+---
```

Call count and data volume of GSPreCon for xSize = 20000.

For **applyStencil** I obtain a total amount of data volume of **584.63 GB** for **45 calls**. Hence, the code balance is:

$$\frac{584.63 \times 10^9\ bytes}{45 \times 399998 \times 998} \approx\ 32 \frac{B}{LUP}$$

Comparing this number with our computation of **40 B/LUP**, it can be said that it is close but not fully accurate.

```
Region APPLY_STENCIL, Group 1: MEM
+-------------------+------------+-
--+-------------+-------------+----
|    Region Info    | HWThread 0 |
2 | HWThread 13 | HWThread 14 | HWT
+-------------------+------------+-
--+-------------+-------------+----
| RDTSC Runtime [s] |   7.634729 |
8 |    7.366277 |    7.385210 |
|     call count    |         45 |
5 |          45 |          45 |
+-------------------+------------+-
```

```
+-----------------------------------+------------+
----+-------------+-------------+-------------+---
|                Metric             | HWThread 0 |
 11 | HWThread 12 | HWThread 13 | HWThread 14 | HW
+-----------------------------------+------------+
----+-------------+-------------+-------------+---
|        Runtime (RDTSC) [s]        |     7.6347 |
437 |      7.3978 |      7.3663 |      7.3852 |
|        Runtime unhalted [s]       |     6.2073 |
901 |      6.1299 |      6.1057 |      6.1223 |
|            Clock [MHz]            |  1995.2627 |
632 |   1995.2609 |   1995.2616 |   1995.2607 |
|                  CPI              |     5.5874 |
005 |      5.8590 |      5.7898 |      6.0813 |
|  Memory read bandwidth [MBytes/s] | 57385.2557 |
  0 |           0 |           0 |           0 |
|  Memory read data volume [GBytes] |   438.1209 |
  0 |           0 |           0 |           0 |
| Memory write bandwidth [MBytes/s] | 19190.8750 |
  0 |           0 |           0 |           0 |
| Memory write data volume [GBytes] |   146.5171 |
  0 |           0 |           0 |           0 |
|    Memory bandwidth [MBytes/s]    | 76576.1307 |
  0 |           0 |           0 |           0 |
|    Memory data volume [GBytes]    |   584.6380 |
  0 |           0 |           0 |           0 |
```

Call count and data volume of **applyStencil** for **xSize = 400000.**

For **GSPreCon** we obtain a total amount of data volume of $\mathbf{335.26}$ GB for **21 calls**. Hence, the code balance is:

$$\frac{335.26 \times 10^9\ bytes}{21 \times 399998 \times 998} \approx\ 40\frac{B}{LUP}$$

Comparing this number with my computation of **56 B/LUP**, is completely different. Hence, this computation might be the reason of my previous wrong prediction. However, recall that for the case **xSize = 20000,** I had an accurate code balance prediction but predicted wrong performance. Whereas in the other case I matched the performance and predicted the wrong code balance.
As in both sizes the memory code balances are similar, I guess that the compiler carries out the write-allocation of **x[j: i]** in the backward substitution and writes it back in the backward substitution. That would mean that 1 LD and 1 ST were extra counted in our computations. Regarding the performance, I cannot clearly figure out where or which part in our analysis was wrong.

```
Region GS_PRE_CON, Group 1: MEM
+-------------------+------------+
--+-------------+-------------+---
|    Region Info    | HWThread 0 |
2 | HWThread 13 | HWThread 14 | HW
+-------------------+------------+
--+-------------+-------------+---
| RDTSC Runtime [s] |   6.225339 |
6 |    6.229767 |    6.229729 |
|     call count    |         21 |
1 |          21 |          21 |
+-------------------+------------+
--+-------------+-------------+---
```

```
+---------------------------------+------------+
----+-------------+-------------+-------------+---
|                Metric                | HWThread 0 |
 11 | HWThread 12 | HWThread 13 | HWThread 14 | HW
+---------------------------------+------------+-
----+-------------+-------------+-------------+---
|        Runtime (RDTSC) [s]        |     6.2253 |
297 |      6.2298 |      6.2298 |      6.2297 |
|        Runtime unhalted [s]       |     5.1597 |
622 |      5.1625 |      5.1618 |      5.1626 |
|             Clock [MHz]           |  1995.2602 |
637 |   1995.2624 |   1995.2632 |   1995.2654 |
|                CPI                |     1.7529 |
375 |      1.7359 |      1.7354 |      1.7334 |
|  Memory read bandwidth [MBytes/s] | 32467.9198 |
  0 |           0 |           0 |           0 |
|  Memory read data volume [GBytes] |   202.1238 |
  0 |           0 |           0 |           0 |
| Memory write bandwidth [MBytes/s] | 21386.4801 |
  0 |           0 |           0 |           0 |
| Memory write data volume [GBytes] |   133.1381 |
  0 |           0 |           0 |           0 |
|    Memory bandwidth [MBytes/s]    | 53854.3999 |
  0 |           0 |           0 |           0 |
|    Memory data volume [GBytes]    |   335.2619 |
  0 |           0 |           0 |           0 |
+---------------------------------+------------+
```

Call count and data volume of **GSPreCon** for **xSize = 400000**.

Part 4: How does the CG and PCG code scale from 1 ccNUMA to one node (4ccNUMA, 72 cores) of Fritz? Does it scale perfectly? If not, why?

Following measurements were taken on a dedicated Fritz node (Intel(R) Xeon(R) Platinum 8360Y CPU v2 @ 2.40GHz) Intel **Icelake SP** processor with a fixed frequency (2GHz) and using pinning (likwid-pin).

ptfs166h@fritz3:~/ptfsProject$ salloc --nodes=1 --time=1:00:00 -p singlenode -C hwperf

Then, I compiled the code:

module load intel
module load likwid
LIKWID =on CXX = icpc make

To measure performance on one socket with setting frequency **2GHz**, I run the binary perf using LIKWID:

ptfs166h@fritz2:~/ptfsProject$ srun --cpu-freq=2000000-2000000 likwid-pin -C S0:0-17 ./perf 2000 20000

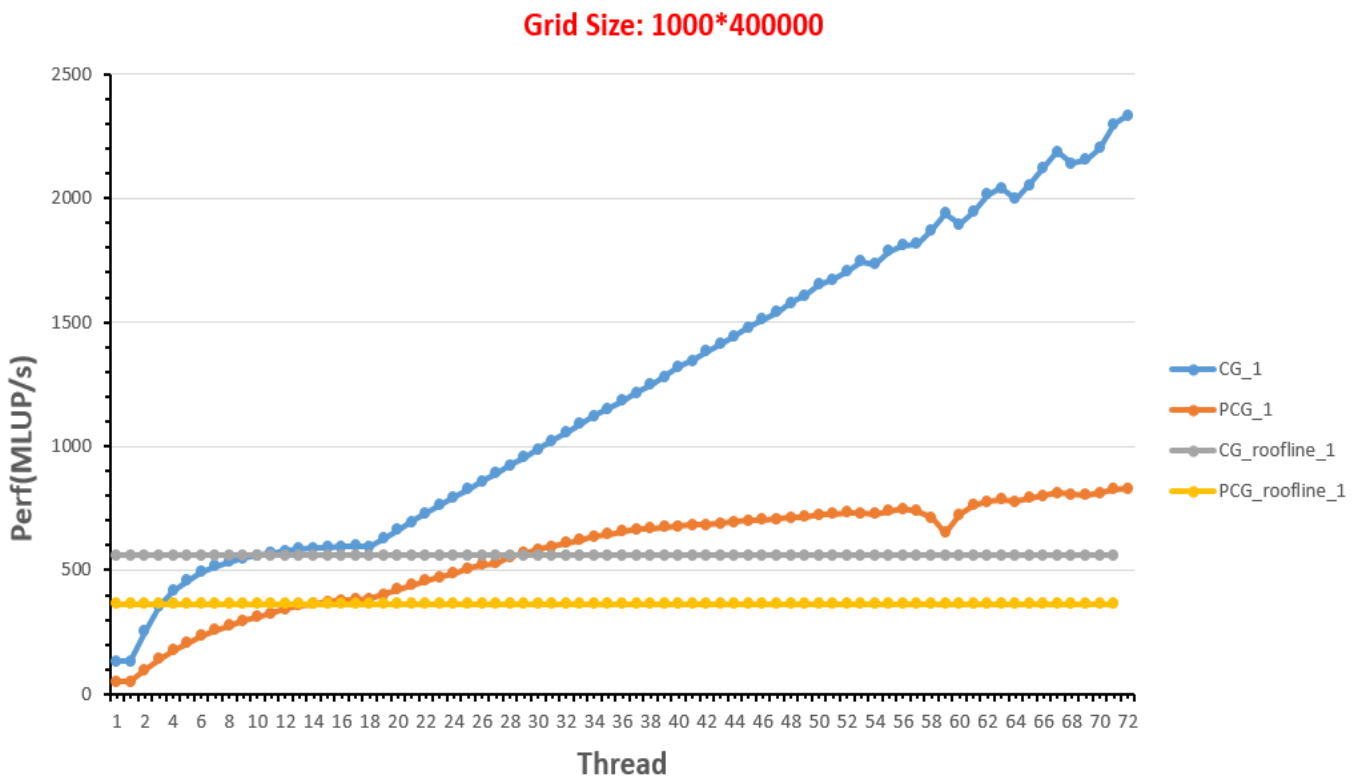
ptfs166h@fritz2:~/ptfsProject$ srun --cpu-freq=2000000-2000000 likwid-pin -C S0:0-17 ./perf 1000 400000

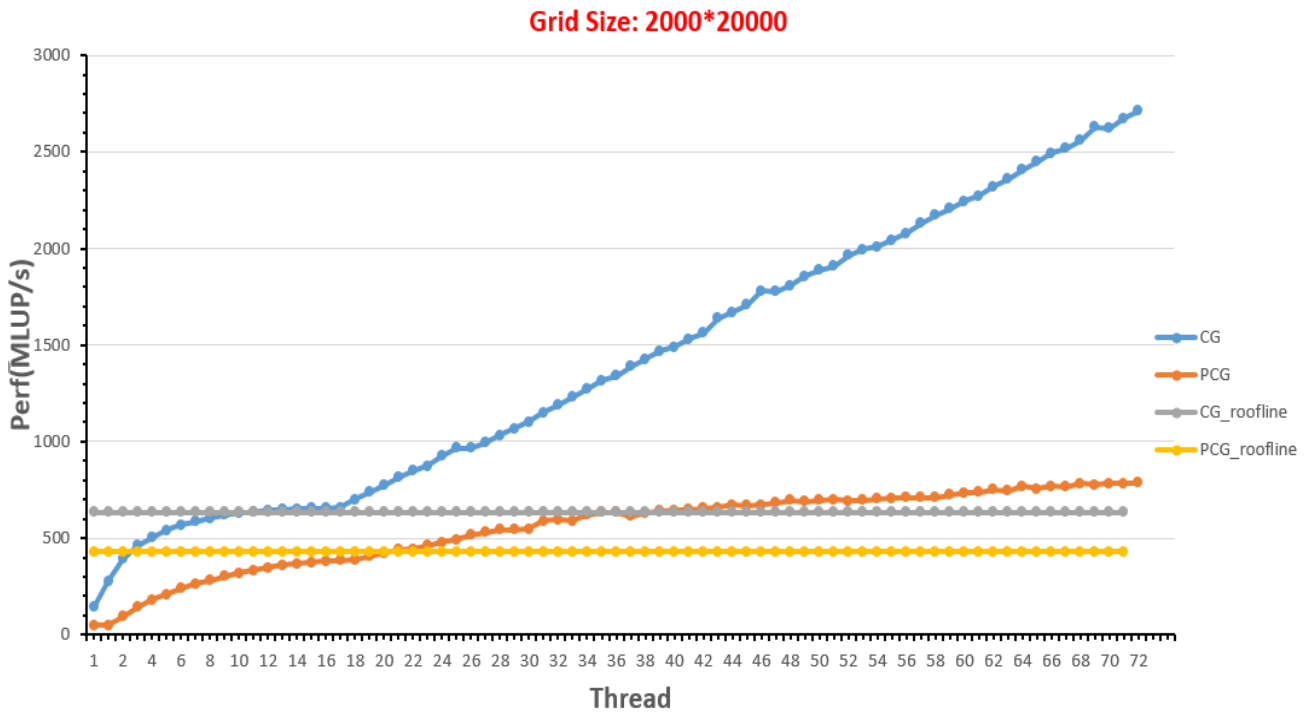
ptfs166h@fritz2:~/ptfsProject$ srun --cpu-freq=2000000-2000000 likwid-pin -C S0:0-71 ./perf 2000 20000

ptfs166h@fritz2:~/ptfsProject$ srun --cpu-freq=2000000-2000000 likwid-pin -C S0:0-71 ./perf 1000 400000

<u>Performance of CG/PCG on 18/72 cores using 2000x20000& 1000x400000 grid:</u>

	18 cores	72 cores	18 cores	72 cores
Grid Size	2000 x 20000		1000 x 400000	
CG (MLUP/s)	658	2710	595	2332
PCG (MLUP/s)	387	786	383	830





Here, I got some overshooting. The parallelization of the initialization routines include the Grid.cpp might have some impact on that. However, it is not enough. There are other possibility that subroutines should have parallelized as well in order to get perfect scalability.